Aran Learn 17: Wiring a Static Design to a Live API

Aran Learn 17: Wiring a Static Design to a Live API

For: everyone who will change the new console in `app/ui`. Inbathamizhan owns the shell and the tour; Aasif owns Customers, Leak trace and the erasure scenes; Arun and Bharadhwaj own the Immune layer, Results and the AI-chatbot chapter. **Time:** about 3 hours with practice. **Before this:** Learn 8 (web APIs) and Learn 11 (HTML, CSS, JavaScript).

The team's design (`ui-design`) began as a static site: every number was invented in the browser so the look could be reviewed with no server. It is now Aran's main console, and every number comes from the real server. This doc teaches the five ideas behind that change: **`fetch` and promises**, **one API layer**, **a live store with polling**, **turning a synchronous mock into an async call**, and **showing errors instead of hiding them**.

1. `fetch`, promises and `await`

`fetch` asks the server for something and returns a **promise**: a box that will hold the answer later.

```
const r = await fetch('/api/stats')          // wait for the reply
const stats = await r.json()                 // wait for the body, parsed as JSON
console.log(stats.requests)
```

- `await` can only be used inside a function marked `async`. An `async` function itself returns a promise.
- `fetch` does **not** throw on HTTP errors like 404 or 503. You must check `r.ok` yourself.
- To send data, use `POST` with a JSON body:

```
await fetch('/api/process', { method: 'POST', headers: { 'content-type':
'application/json' },
  body: JSON.stringify({ text: 'call me on 9876543210', source: 'console', sink:
'crm-notes' }) })
```

2. One API layer: `A.api`

Every page in `app/ui` talks to the server through one helper in `js/data.js`:

```
A.api = async function (path, body, method) {
  const opt = body === undefined ? { method: 'GET' } : { method: 'POST', headers: {
'content-type': 'application/json' }, body: JSON.stringify(body) }
  const r = await fetch('/api' + path, opt)
  const d = await r.json().catch(() => null)
  if (!r.ok) throw new Error(d && typeof d.detail === 'string' ? d.detail : 'HTTP '
+ r.status)
```

```
  return d
}
A.checksum = (digits) => A.api('/tour/checksum', { digits })
A.erase = (ref, purpose) => A.api('/erase', { customer_ref: ref, purpose })
```

Why one layer: the server's error message (FastAPI puts it in `detail`) becomes a normal JavaScript `Error`, so every page shows the same, readable message, for example "KYC records are under a retention lock until 15 Oct 2031". If an endpoint ever moves, only one line changes.

3. A live store with polling

Some data is shown on many pages at once: statistics, recent events, alerts. Instead of every page fetching it, `data.js` keeps a **store** and refreshes it on a timer:

```
A.stats = {}; A.events = []
A.refresh = async (...keys) => { /* fetch each key, then tell whoever subscribed */
}
A.poll = () => { setInterval(() => A.refresh('stats', 'events'), 3000) }
const off = A.on('stats', () => redraw())   // subscribe; call off() to stop
```

- **Polling** means "ask again every few seconds". It is simple and good enough for a console; the alternative (WebSockets, server-sent events) pushes updates but needs more server code.
- **Always clean up.** Every page's `mount()` returns a cleanup function. The router calls it when you leave the page, so timers and subscriptions stop. Forget this and old pages keep fetching in the background.
- **Boot first.** `A.boot()` polls `GET /api/health` until the models are loaded, shows the "Starting Aran" screen meanwhile, then fills the store. Only then does the first page draw.

4. Turning a mock into a real call

The design's mocks were synchronous: `const r = A.checksum(v)` returned the answer at once. A real call returns a promise, so three things change:

```
// before (mock)
const run = (v) => {
  const r = A.checksum(v)
  draw(r)
}
```

```
// after (live)
const run = async (v) => {
  let r
  try { r = await A.checksum(v) }
  catch (e) { show(U.note('err',
e.message)); return }

  draw(r)
}
```

1. The function becomes `async` and the call gets `await`.
2. The call can now fail, so wrap it in `try` / `catch` (the tour uses a small `guard()` helper for this).
3. The page may have changed while you waited. Long-running scenes keep an `alive` flag that cleanup sets to `false`, and check it before drawing.

The shapes did not change: the design's mock data was written from the API spec, so `r.steps`, `r.valid` and `r.correct_check_digit` mean the same thing live. When a shape did differ (the results files nest their numbers under `data`), one mapping function (`mapResults`) converts it in `data.js`, and the pages stay unchanged.

5. Never fake, never hide

- **No invented numbers on a real page.** If there is nothing to show yet (no traffic, no alerts, no result files), say so and say how to get it: "No Aadhaar has gone through this server yet. Send a bank request."
- **Show the server's error where the user is looking**, next to the button they pressed, not only in the console.
- **Let the server decide.** The browser still has Verhoeff and Luhn code, but only to colour an input while typing. Whether a number is valid, whether a receipt verifies, which sink leaked: those answers come from the server. The one check the browser repeats on purpose is the Merkle inclusion proof of an erasure receipt, so a viewer does not have to trust the server's word.

Practice

1. **Fetch.** In the browser console on http://localhost:8000, run `await (await fetch('/api/stats')).json()` and find `values_protected`.
2. **Use the layer.** Run `await ARAN.checksum('234123412346')`, then `await ARAN.checksum('123')` inside a `try` / `catch` and print the error message.
3. **Subscribe.** Run `const off = ARAN.on('stats', () => console.log(ARAN.stats.requests))`, send a request from Live traffic, watch the number change, then call `off()`.
4. **Convert a mock.** Write a synchronous function `fakeGenerate()` that returns `{ aadhaar: '...' }`, then change one caller to use `await ARAN.generate()` instead, with error handling.
5. **Clean up.** Open the Immune layer page, then another page. In the Network tab, check that `/api/tour/immune-radar` stops being requested.

Solutions

1. The object printed has `values_protected` near the top; it grows each time a request is protected.
2. `try { await ARAN.checksum('123') } catch (e) { console.log(e.message) }` prints "enter 12 digits (Aadhaar) or 16 digits (card)", the server's own message.
3. A number is printed about every 3 seconds; after `off()` nothing more is printed for that subscription.
4. `const run = async () => { try { const g = await ARAN.generate(); use(g.aadhaar) } catch (e) { show(e.message) } }`.
5. The Immune page's `mount()` returns `() => { alive = false; clearInterval(timer) }`, which the router calls on navigation, so the requests stop.